

Technical Requirements Document (TRD) – Asset Maintenance Tool

****Version:**** 1.0.0

****Status:**** Draft – pending review

****Last updated:**** 2026-08-15

****Owner:**** Founding Product Architect

****Stable requirement IDs:**** TRF-*, TR-NFR-*, TR-RBAC-*

1. Purpose

This document captures the detailed technical architecture, API contracts, data model, security model, infrastructure, and operational requirements for the Asset Maintenance Tool SaaS. It is intended for engineers, architects, and technical reviewers. The companion Product Requirements Document (PRD-v1.0.0.md) focuses on user problems, goals, features, and business metrics.

All technical decisions shall be traceable to this document via stable IDs (TRF-*, etc.) and shall be validated by corresponding tests.

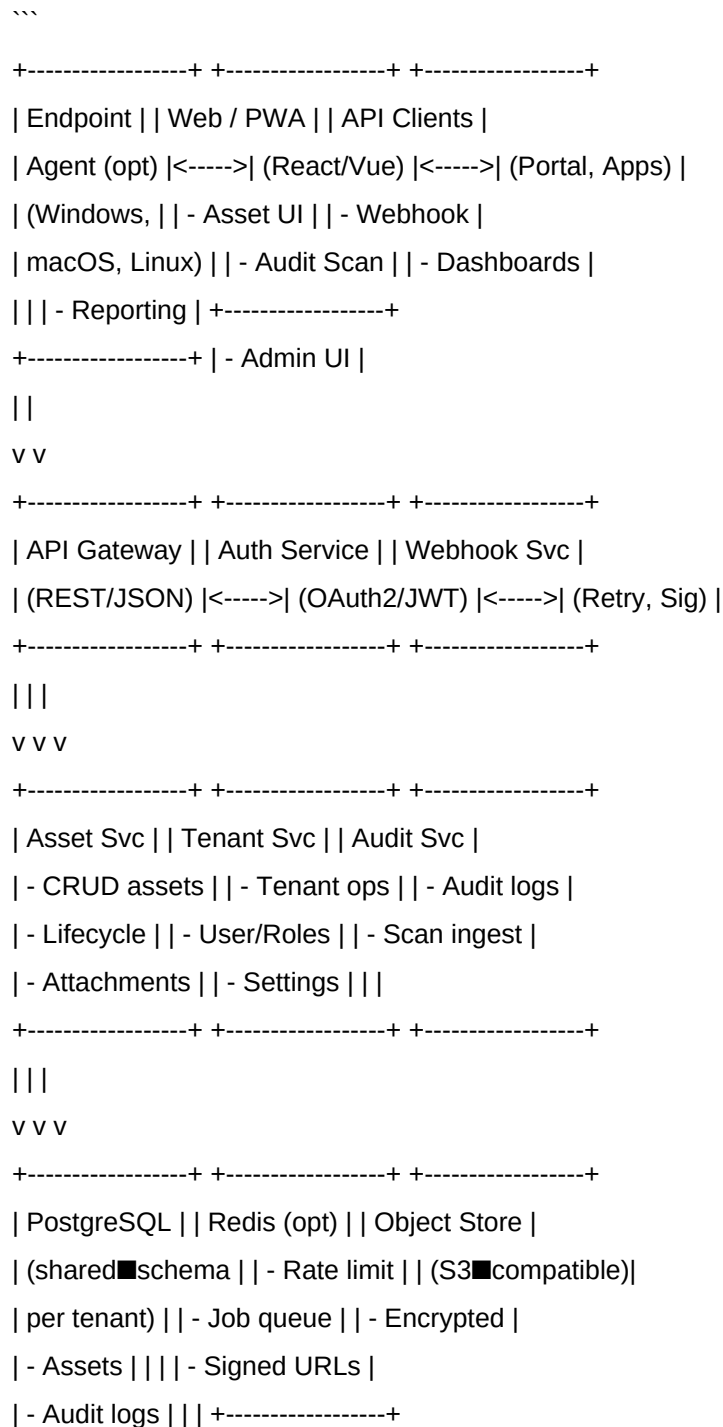
2. System Overview

The Asset Maintenance Tool is a multi-tenant SaaS that provides:

- A responsive web application (progressive web app) for asset lifecycle management.
- A versioned REST/JSON API (v1) for programmatic access and integrations.
- An optional lightweight endpoint agent for continuous hardware/software inventory on employee laptops.
- Background workers for imports, exports, report generation, notifications, and label printing.
- Secure object storage for attachments, imports/exports, and backups.
- A relational database (PostgreSQL-compatible) for asset records, audit logs, users, permissions, and configuration.
- Observability stack: structured logs, Prometheus metrics, OpenTelemetry tracing.
- Automated build, test, and deployment pipelines with zero-downtime release strategies.

The system enforces ****tenant isolation**** at every layer: network, application, API, and data.
Cross-tenant access is blocked by default.

3. Architecture Diagram (Textual)



```

| - Users | | |
| - Permissions | | |
+-----+ +-----+ +-----+
| | |
v v v
+-----+ +-----+ +-----+
| Observability | | Background | | CI/CD Pipeline |
| (Logs, Metrics, | | Workers | | (Build, Test, |
| Traces) | | - Imports | | Deploy) |
| - Loki/Prometheus | | - Exports | | |
| - Tempo/OpenTelemetry | | - Reports | | |
+-----+ +-----+ +-----+
...

```

4. Tenant Isolation & Data Model

4.1 Tenant Strategy

We adopt a ****shared database, shared schema**** approach with a `tenant_id`` column on every tenant-owned table. Row-Level Security (RLS) policies enforce that queries automatically filter by the current tenant's ID (derived from JWT/session). Alternative: separate schemas per tenant if regulatory isolation is required (configurable via deployment variable).

4.2 Core Entities (Simplified)

```

| Entity | Key Fields | Tenant-Scoped? | Notes |
|-----|-----|-----|-----|
| `tenants` | id, name, domain, plan, created_at | N/A | Top-level container |
| `users` | id, tenant_id, email, role, status, mfa_enforced | Y | Linked to tenant |
| `assets` | id, tenant_id, asset_tag_norm, serial_no, make, model, category_id, site_id, location_id, department_id, custodian_user_id, custodian_group_id, status, purchase_date, cost, warranty_expiry, created_at, updated_at | Y | `asset_tag_norm` has unique index per tenant |
| `asset_tags` (history) | id, asset_id, tag_value, changed_at, changed_by | Y | Audit trail of tag changes |
| `sites` | id, tenant_id, name, address, created_at | Y | Hierarchical: sites > locations |

```

| `locations` | id, tenant_id, site_id, name, description, created_at | Y | |

| `categories` | id, tenant_id, name, description, created_at | Y | For asset grouping |

| `departments` | id, tenant_id, name, description, created_at | Y | For cost center |

| `custodian\_groups` | id, tenant_id, name, description, created_at | Y | Optional grouping of users |

| `asset\_events` | id, tenant_id, asset_id, event_type (check_out, check_in, maintenance_start, maintenance_complete, reserve, dispose, retire), occurred_at, recorded_by, notes | Y | Immutable audit of lifecycle |

| `audit\_sessions` | id, tenant_id, started_by, started_at, ended_at, status, notes | Y | |

| `audit\_session\_items` | id, session_id, asset_tag_scanned, result (found, missing, mismatched, damaged), scanned_at, notes | Y | |

| `attachments` | id, tenant_id, asset_id, file_name, mime_type, size_bytes, sha256, storage_path, uploaded_at, uploaded_by | Y | Encrypted storage, malware scan flag |

| `webhooks` | id, tenant_id, url, secret, events (list), active, created_at | Y | Outbound webhook config |

| `api\_keys` | id, tenant_id, name, key_hash, scopes, active, created_at, last_used_at | Y | For machine-to-machine auth |

| `settings` | id, tenant_id, key, value, description, updated_at | Y | Tenant-wide configuration (e.g., label printer, notification prefs) |

All tables have `created\_at`, `updated\_at` (where appropriate) and `deleted\_at` (soft delete if needed).

4.3 Indexes & Constraints

- Primary keys: UUIDv7 (or ULID) for distributed generation, or BIGINT with sequence.
- Unique index: `(tenant\_id, normalized\_asset\_tag)` on `assets`.
- Foreign keys with `ON DELETE CASCADE` or `SET NULL` as appropriate.
- Check constraints: `status` enum, `cost >= 0`, `warranty\_expiry >= purchase\_date`.
- Triggers: set `updated\_at` on update, populate `asset\_tag\_norm` via normalization function (trim, upper, allow alnum+hyphen+underscore).

5. API Contracts

5.1 General

- **Protocol:** HTTPS only, TLS 1.2+.
- **Authentication:**
- Session cookie (HttpOnly, Secure, SameSite=Strict) for web UI.

- Bearer token (JWT signed by internal Auth Service or OAuth2 access token) for API clients and agents.
- API Key header (`X-API-Key`) for service-to-service integrations (optional).
- **Versioning:** Path-based, e.g., `/api/v1/assets`. Minor version increments are backward compatible; major versions may break.
- **Content-Type:** `application/json` for request and response bodies.
- **Character Encoding:** UTF-8.
- **Errors:** RFC 7807 Problem Details JSON (`type`, `title`, `status`, `detail`, `instance`).
- **Pagination:** Cursor-based (`limit`, `cursor`) for large collections; offset-based allowed for small lists (< 1000 rows).
- **Sorting:** `sort=field:desc` syntax, multiple fields allowed.
- **Filtering:** Query parameters per resource (e.g., `?status=Assigned&location;\_id=...`).
- **Idempotency:** Mutating endpoints support `Idempotency-Key` header; replay returns same response (status 200/201) without side effect.
- **Concurrency:** Optimistic locking via `version` column (integer) on mutable assets; `409 Conflict` if version mismatch.
- **Rate Limiting:** Per-tenant, per-API-key, or per-IP basis (configurable); returns `429 Too Many Requests` with `Retry-After`.
- **Bulk Operations:**
 - Import: `POST /api/v1/assets/import` accepts multipart/form-data (CSV/JSON) with preview step.
 - Export: `GET /api/v1/assets/export?...` returns CSV/JSON stream; honors filters, pagination, and role-based field masking.
- **Webhooks:**
 - Configuration: `POST /api/v1/webhooks` (tenant admin).
 - Delivery: HTTP POST to registered URL with JSON body and `X-Signature` header (HMAC-SHA256 using secret).
 - Retry: Exponential backoff (1 s, 2 s, 4 s, 8 s, 16 s) up to 5 attempts; dead-letter after final failure.
- **Search:** Full-text search on asset description, make, model, serial via PostgreSQL `tsvector` or dedicated service (optional, later phase).

5.2 Endpoint Summary (Illustrative)

Method	Path	Description	Auth	Idempotent?
-----	-----	-----	-----	-----
GET	`/api/v1/assets`	List assets with filtering, pagination, sorting	Session/JWT/API-Key	Yes
POST	`/api/v1/assets`	Create asset (requires unique tag)	Session/JWT/API-Key	Yes (with Idempotency-Key)
GET	`/api/v1/assets/{asset_id}`	Get single asset	Session/JWT/API-Key	Yes

| PUT | `/api/v1/assets/{asset\_id}` | Full replace asset | Session/JWT/APIKey | Yes (with Idempotency-Key) |

| PATCH | `/api/v1/assets/{asset\_id}` | Partial update asset | Session/JWT/APIKey | Yes (with Idempotency-Key) |

| DELETE | `/api/v1/assets/{asset\_id}` | Softdelete asset (set deleted_at) | Session/JWT/APIKey | Yes |

| POST | `/api/v1/assets/{asset\_id}/check-out` | Check out asset to custodian | Session/JWT | Yes |

| POST | `/api/v1/assets/{asset\_id}/check-in` | Check in asset (assess condition) | Session/JWT | Yes |

| POST | `/api/v1/assets/{asset\_id}/maintenance/start` | Start maintenance work order | Session/JWT | Yes |

| POST | `/api/v1/assets/{asset\_id}/maintenance/complete` | Complete maintenance, set condition | Session/JWT | Yes |

| POST | `/api/v1/audit/sessions` | Start audit session (specify scope) | Session/JWT | Yes |

| POST | `/api/v1/audit/sessions/{session\_id}/scans` | Record barcode scan result | Session/JWT | Yes |

| POST | `/api/v1/audit/sessions/{session\_id}/complete` | Finish audit session, trigger reconciliation | Session/JWT | Yes |

| GET | `/api/v1/reports/assets-by-custodian` | Generate CSV report (streaming) | Session/JWT | Yes |

| POST | `/api/v1/notifications/test` | Send test notification (email/SMS/webhook) | Session/JWT (admin) | Yes |

| POST | `/api/v1/webhooks/{webhook-id}/test` | Send test webhook payload | Session/JWT (admin) | Yes |

| POST | `/api/v1/agent/ingest` | Receive data from endpoint agent (mTLS or JWT) | AgentAuth (mTLS/JWT) | Yes (deduplicate by lastseen) |

Full OpenAPI 3.0 specification will be generated from the source of truth (`openapi.yaml`) and stored in `docs/api/openapi/v1.0.0.yaml`.

6. Security Model

6.1 Authentication

- **Passwords:** Argon2id (min 64 MiB RAM, 3 iterations), minimum length 12, breachcheck via HaveIBeenPwned API (daily).
- **MultiFactor Authentication (MFA):** TOTP (Google Authenticator compatible), push (via Duo/Authy), hardware token (U2F/WebAuthn). Enforced for admins; optional for end users (configurable per tenant).
- **Single SignOn (SSO):** SAML 2.0 and OpenID Connect (OIDC) integrations with Azure AD, Okta, Google Workspace, JumpCloud. Justintime (JIT) user provisioning supported.

- **Session Management:** Random 128-bit session ID, stored encrypted server-side, absolute timeout 15 minutes, idle timeout 5 minutes. Cookie attributes: HttpOnly, Secure, SameSite=Strict.
- **API Key:** 32-byte cryptographically random string, hashed with SHA-256 before storage; scopes limit accessible endpoints.

6.2 Authorization (RBAC)

- **Default deny:** All API endpoints start with `deny`; an explicit `allow` rule must match the caller's role, tenant, resource, and action.
- **Roles (built-in):**
 - `super\_admin`: Full access to all tenants (platform level).
 - `tenant\_admin`: Full access within a tenant (billing, settings, users, etc.).
 - `it\_asset\_manager`: Full asset lifecycle, audit, reports, exports.
 - `field\_technician`: View assets, start/finish maintenance, scan barcodes, update work orders.
 - `employee\_end\_user`: Self-service portal: view own assets, request assets, report issues, initiate check-in/out.
 - `auditor`: Read-only access to assets, audit logs, reports (no mutation).
 - `read\_only`: View assets only (no audit, no reports).
- **Custom Roles:** Administrators may create roles by selecting granular permissions (e.g., `asset:create`, `asset:delete`, `audit:start`, `report:export`, `webhook:manage`).
- **Permission Granularity:** Resources: `asset`, `asset\_tag`, `audit\_session`, `report`, `webhook`, `api\_key`, `setting`, `user`, `group`, `tenant`. Actions: `create`, `read`, `update`, `delete`, `export`, `import`, `manage`.
- **Policy Storage:** Stored as JSON blobs in `role\_permissions` table; evaluated at runtime via rule engine (e.g., Casbin-like) or simple SQL joins.

6.3 Data Protection

- **Encryption at Rest:**
 - Relational database: Transparent Data Encryption (TDE) via cloud provider or pgcrypto.
 - Object storage: Server-Side Encryption (SSE-S3 or SSE-KMS) with customer-managed keys where available.
 - Backups: Encrypted using same KMS key; retained per policy.
- **Encryption in Transit:** TLS 1.2+ everywhere; internal service-to-service may use mTLS or JWT signed by internal auth service.
- **Secrets Management:** HashiCorp Vault or cloud provider secret manager (AWS Secrets Manager, Azure Key Vault) for database passwords, API keys, webhook secrets, encryption keys. Rotation every 90 days.

- **Malware Scanning:** All file uploads (attachments, imports) scanned via ClamAV latest definitions; upload rejected if threat found. Definitions updated hourly.
- **File Type Whitelist:** Attachments limited to `.pdf`, `.jpg`, `.jpeg`, `.png`, `.docx`, `.xlsx`, `.csv`, `.zip` (max 25 MB per file).
- **Signed URLs:** Object storage returns time-limited (e.g., 15 minutes) signed URLs for download; direct bucket access is blocked.
- **Audit Logging:** Security-relevant events (login success/failure, permission changes, role assignments, data export, API key creation, webhook firing) written to immutable append-only table (`security\_audit\_log`) with tamper-evident hashing (SHA-256 chain) and regular off-site copy.

6.4 Vulnerability Management

- Dependencies scanned via `npm audit`, `pip-audit`, `cargo audit`, `mvn dependency:check` in CI.
- Container images scanned with Trivy or Clair.
- Periodic dependency updates (monthly) with automated PRs.
- Annual third-party penetration test and bug bounty program (scope: web UI, API, agent).
- Security headers: `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, `X-XSS-Protection: 1; mode=block`, `Referrer-Policy: strict-origin-when-cross-origin`, `Content-Security-Policy: default-src 'self'; img-src \* data;; style-src 'self' 'unsafe-inline';`.

7. Non-Functional Requirements (Technical Focus)

| ID | Description |

|----|-----|

| TR-NFR-PERF-001 | 95 % of API requests (asset CRUD, audit scan ingest) shall respond ≤ 200 ms at 95th percentile under a load of 200 RPS with 400 concurrent users (simulated). |

| TR-NFR-PERF-002 | Bulk import of 10 000 assets (CSV) shall complete within 90 seconds and not increase 95th-percentile latency of foreground requests by > 5 %. |

| TR-NFR-SCAL-001 | The system shall support horizontal scaling of stateless API nodes (≥ 3 behind LB) and background worker pools (≥ 2 workers per job type). |

| TR-NFR-AVAIL-001 | Monthly uptime SLA: 99.9 % (excluding scheduled maintenance windows with ≥ 24 h notice). |

| TR-NFR-MOB-001 | The progressive web app shall achieve Lighthouse performance ≥ 90 and accessibility ≥ 85 on mid-tier mobile (Moto G4 Powertrain) for core audit flow. |

| TR-NFR-OFFL-001 | The PWA shall be usable offline for audit scans: cached asset list (ID, tag, last known location), ability to store scans locally, and sync on reconnection with conflict-resolution (last write wins). |

| TR-NFR-BACKUP-001 | Daily encrypted snapshot of database (logical dump or physical snapshot) and object storage metadata; retained 30 days; restore test performed quarterly; RPO $\leq$ 15 minutes, RTO $\leq$ 60 minutes. |

| TR-NFR-OBSERV-001 | Structured logs (JSON) emitted to stdout; Prometheus metrics endpoint (`/metrics`) exposes HTTP request latency, error rates, DB query times, job queue lengths; OpenTelemetry traces exported to Jaeger/Tempo. |

| TR-NFR-LEGAL-001 | If tenant elects EU data residency, deployment shall be in an Azure/AWS EU region; all personal data remains within that region. |

| TR-NFR-LEGAL-002 | The system shall provide a Data Processing Addendum (DPA) template for GDPR/CCPA compliance upon request. |

| TR-NFR-MAINT-001 | Deployments shall use blue/green or canary strategy with automated rollback on health check failure (error rate $> 1\%$ or latency $> 2\times$ baseline for 30 s). |

| TR-NFR-MAINT-002 | Schema migrations shall be backward compatible (additive columns, nullable defaults) or provide explicit up/down scripts; no migration requiring downtime > 5 minutes. |

| TR-NFN-INT-001 | All external integrations (webhooks, API keys) shall support mutual TLS or signed JWT with configurable expiry and rotation. |

| TR-NFN-SEC-001 | Automated nightly authorization test suite shall attempt every role/resource/action combination and assert expected status; any failure blocks promotion to production. |

| TR-NFN-SEC-002 | Dependency check job shall run weekly; any critical (CVSS ≥ 7.0) vulnerability found shall trigger a patch release within 48 hours. |

8. Communication & Protocols

8.1 Internal Service Communication

- **REST/JSON** over HTTPS with mutual TLS or JWT bearer tokens issued by internal Auth Service (short-lived, 5 min).

- **gRPC** may be used for high-volume internal streams (e.g., event streaming) – optional, later phase.

- **Message Queue**: Redis Pub/Sub or PostgreSQL `LISTEN/NOTIFY` for lightweight notifications; for durable queues, use PostgreSQL-based job table or external Redis/RabbitMQ if throughput demands.

8.2 External Protocols

- **Webhook Delivery**: HTTP POST with JSON body, `Content-Type: application/json`, and `X-Signature: t=timestamp,v1=hexhmac` (HMAC-SHA256).

- **Inbound Webhooks** (if ever offered): Same as outbound but with verification; rate limited per tenant.

- **SMTP/Email:** TLS, STARTTLS, authentication via username/password or API key (SendGrid, SES, Mailgun).
- **SMS:** HTTP JSON API to provider (Twilio, Nexmo, Plivo) with API key.
- **Syslog:** Optional forwarding of structured logs to external SIEM (TLS).

8.3 Endpoint Agent Protocol

- **Transport:** Mutual TLS (mTLS) with client certificate issued by tenant-specific CA, or Bearer JWT (short-lived, signed by internal auth service) over HTTPS.
- **Message Format:** Protobuf or JSON; fields: ``asset_tag_raw``, ``installed_software[]``, ``usage_stats``, ``hardware_snapshot``, ``os_info``, ``network_status``, ``security_status``, ``timestamp``.
- **Frequency:** Configurable per tenant (default 900 seconds).
- **Backoff:** Exponential with jitter on failure; max interval 1 hour.
- **Privacy:** Agent does not collect personal files, keystrokes, camera/microphone, or screen captures. Only system-level telemetry as above.
- **Updates:** Agent binary signed; update mechanism checks signed manifest from update server (tenant-specific or global) and verifies signature before replacing binary.

9. Observability & Monitoring

9.1 Logging

- **Format:** JSON lines, fields: ``timestamp``, ``level``, ``trace_id``, ``span_id``, ``service``, ``msg``, plus context (``user_id``, ``tenant_id``, ``asset_id``, ``request_id``, etc.).
- **Sources:** API gateway, auth service, asset service, audit service, webhook worker, background jobs, endpoint agent ingress.
- **Storage:** Local file rotation → forwarded to Loki or Elasticsearch; retained 30 days.
- **Alerting:** Rules on ``log_level=ERROR`` rate > 5/min, ``http_request_duration_seconds{code=5xx}`` > 0.1/min, ``auth_failed_logins_total`` > 10/min.

9.2 Metrics

- **Prometheus Endpoint:** ``/metrics`` (scraped every 15 s).
- **Counters:** ``http_requests_total{method,endpoint,code,tenant}``, ``asset_creates_total``, ``asset_updates_total``, ``asset_deletes_total``, ``audit_scans_total``, ``webhook_attempts_total``, ``webhook_failures_total``, ``agent_ingest_total``, ``agent_ingest_failures_total``.

- **Histograms:** ``http_request_duration_seconds`, `db_query_duration_seconds`, `job_processing_duration_seconds`.`
- **Gauges:** ``tenant_asset_count`, `tenant_user_count`, `tenant_webhook_count`, `agent_online_count`.`

9.3 Tracing

- **Instrumentation:** OpenTelemetry SDK in all services.
- **Exporter:** OTLP over HTTP to Tempo or Jaeger.
- **Trace Context Propagation:** W3C TraceContext header.
- **Key Traces:** User login, asset checkout, audit session start/complete, webhook delivery, agent ingest.

10. Development & Release Process

10.1 Source Control

- **Monorepo:** Single Git repository containing:
 - Infrastructure as Code (Terraform) under ``infra/``
 - Backend source (e.g., Go, Python, Node.js) under ``backend/``
 - Frontend source (React/TypeScript) under ``frontend/``
 - Endpoint agent source (C#/.NET or Rust) under ``agent/``
 - OpenAPI spec under ``docs/api/``
 - Database migrations under ``migrations/``
- **Branch Protection:** ``main`` requires PR approval, mandatory status checks (unit, integration, security, license scan), and linear history (no merge commits; squash merge preferred).

10.2 CI Pipeline

Triggered on PR and push to ``main``:

1. **Lint:** ESLint, Prettier, golangci-lint, etc.
2. **Unit Tests:** Jest, pytest, Go test – coverage ≥ 80 %.
3. **Integration Tests:** Spin up compose stack (Postgres, Redis, MinIO, mock SMTP) and run API contract tests (Newman or Pact).
4. **Security Scan:**

- OWASP ZAP baseline scan on staging UI.
 - Dependency audit (`npm audit`, `pip audit`, `cargo audit`).
 - Secret detection (`git secrets`, `truffleHog`).
 - Container image scan (Trivy).
5. **License Check:** Verify all dependencies are permissive (MIT, Apache 2.0, BSD); GPL 3.0+ requires review.
 6. **Build Artifacts:** Docker images for backend, frontend, agent; pushed to internal registry.
 7. **Deploy to Staging:** Auto deploy to preview environment; run smoke tests.

10.3 CD Pipeline (Manual Approval)

After PR merged to `main`:

1. **Manual Gate:** Product manager approves release to production.
2. **Blue-Green Deployment:**
 - Deploy new version to idle slot (`green`).
 - Run health checks (`/healthz`, `/ready`) and synthetic transactions.
 - If healthy, switch router to `green`; old version becomes `blue`.
 - Monitor 5 minutes; if error rate > 1 % or latency spikes, roll back.
3. **Post-Deploy:**
 - Run database migrations (if any) – additive only.
 - Smoke test critical flows (login, asset list, checkout, audit start).
 - Notify stakeholders via Slack/email.
4. **Rollback Procedure:**
 - If health check fails, switch router back to `blue` and terminate `green`.
 - If data migration was applied and rollback needed, execute forward fix or restore from latest backup (RPO ≤ 15 min, RTO ≤ 60 min).

10.4 Feature Flags

- Launch Darkly or Homegrown service (e.g., LaunchDarkly, Unleash, or custom feature flag table) to gate new functionality behind toggles.
- Enable for internal dogfood, then % of tenants, then 100 %.

11. Compliance & Auditing

11.1 Audit Log

- Immutable append-only table (`security\_audit\_log`) capturing:
 - Authentication events (login success/failure, MFA challenge).
 - Authorization changes (role assignments, permission updates).
 - Data exports (asset list, reports).
 - API key creation / rotation.
 - Webhook configuration changes.
 - GDPR deletion requests.
- Each entry signed via hash-chain (`prev\_hash || entry\_data`) and stored with SHA-256.
- Nightly copy to write-once object storage (WORM bucket) for long-term retention.

11.2 Data Subject Requests (DSR)

- **Right to Access:** Provide JSON export of all personal data linked to a user (name, email, login history, asset custody history, etc.).
- **Right to Rectification:** Allow update of incorrect personal data (via UI or API).
- **Right to Erasure:** Upon request, anonymize personal data (replace with pseudonyms) and remove audit trail linking to the user (or retain only aggregated statistics).
- **Right to Portability:** Provide machine-readable JSON/CTV of asset data the user owns or manages.
- **Objection & Automated Decision-Making:** Log any profiling; offer opt-out.

11.3 Reports & Certifications

- SOC 2 Type II (planned for year 2).
- ISO 27001 (planned).
- CSA STAR Level 2.
- Penetration test report (annual).
- Vulnerability disclosure policy (public page).

12. Deployment Topology (Example – Azure)

Component	Azure Service	Size / Notes
-----	-----	-----

| Resource Group | `rg-assetmt-prod-[tenantID]` | – |

| Frontend (PWA) | Azure Static Web Apps | Custom domain, HTTPS |

| API Gateway | Azure API Management | Basic tier, subscription-based auth |

| Asset Service | Azure App Service (Linux) | P1v2, 2 instance scale set |

| Auth Service | Azure App Service (Linux) | P1v2, 2 instances |

| Background Workers | Azure Functions (Consumption plan) | Triggered by Queue/Timer |

| Database | Azure Database for PostgreSQL Flexible Server | Burstable B2s, 2 vCore, 8 GB RAM, zone-redundant, private endpoint |

| Object Storage | Azure Blob Storage (Hierarchical Namespace) | Hot tier, private endpoint, blob versioning |

| Cache (optional) | Azure Cache for Redis | Standard C1, 1 GB |

| Message Queue (optional) | Azure Storage Queues | – |

| Observability | Azure Monitor + Application Insights | Logs, metrics, distributed tracing |

| Key Vault | Azure Key Vault | Stores DB passwords, API keys, webhook secrets |

| Container Registry | Azure Container Registry | Geo-replicated, webhook-enabled |

| CI/CD | Azure DevOps Pipelines | YAML pipelines, self-hosted Linux agents |

| Endpoint Agent Update Store | Azure Blob Storage (public read, signed URLs) | Agent manifests and binaries |

Equivalent AWS/GCP topologies exist.

13. Risks & Mitigations (Technical)

Risk	Likelihood	Impact	Mitigation
Tenant-level data leak via RLS misconfiguration	Low	Critical	Automated nightly tests that attempt cross-tenant queries using fabricated JWTs; enforce defense-in-depth (application checks + DB RLS).
Agent installation blocked by corporate IT policy	Medium	High	Offer agentless mode (periodic CSV import) and provide MSI/package signed by trusted corporate CA; publish source for review.
Webhook replay attack	Low	Medium	Use timestamp (`t`) in signature and reject if > 5 min skew; use nonce cache (Redis) to detect duplicates within window.
Disk full due to uncontrolled log growth	Low	Medium	Structured logs rotated; external shipping to Loki/S3 with lifecycle policy; alert on disk > 80 %.
Dependency supply-chain attack	Low	High	Lockfile-checked builds; SBOM generation; signed artifacts; use internal proxy for npm/pypi/crates.
Schema migration lockout on large table	Low	Medium	Prefer additive columns; if backfill needed, do in batches with `pg_repack` or similar; maintain up/down scripts.

| Cloud provider regional outage | Low | High | Multi-region active-passive standby (active-active possible with conflict-free replicated data – later); automated failover via health checks. |

| LDAP/SSO misconfiguration leading to lockout | Low | Medium | Keep fallback local admin account (break-glass) with MFA-protected emergency access. |

| Invalid SSL certificate causing browsers to block | Low | Medium | Automated cert renewal (Let's Encrypt or cloud provider managed); monitor expiry via alert. |

14. Open Technical Decisions (Require Client Approval)

The following items are ****PROPOSED**** in this TRD and need explicit client approval before becoming binding:

1. ****Database Tenancy Model**** – Choose between shared-schema with `tenant\_id` column (default) or separate schemas per tenant (higher isolation, higher ops overhead). Impact on backup/restore, cross-tenant queries, and cost.
2. ****Background Job Engine**** – Use database-based job table with advisory locks (simple, no extra service) vs. external Redis-based queue (higher throughput, adds dependency). Impact on operational complexity and failure handling.
3. ****Message Bus for Events**** – Adopt Apache Kafka or Redis Streams for event-driven architecture (e.g., asset lifecycle events → downstream services) vs. simple synchronous calls. Impact on scalability, event ordering, and cost.
4. ****Frontend Framework**** – Choose React with TypeScript (current plan) vs. Svelte or Vue.js. Impact on bundle size, learning curve, and available component libraries.
5. ****Endpoint Agent Language**** – Implement agent in .NET 6 (C#) for Windows first, then cross-platform via .NET MAUI, or use Rust for a single binary across all OSes. Impact on development speed, binary size, and maintenance.
6. ****Secret Management Tool**** – Use cloud provider's native secret manager (AWS Secrets Manager / Azure Key Vault) vs. HashiCorp Vault (more flexible, adds operational overhead). Impact on credential rotation and audit integration.
7. ****Observability Backend**** – Use Loki + Prometheus + Tempo (Grafana OSS) vs. Elasticsearch APM or Datadog (commercial, higher cost). Impact on query flexibility and licensing.
8. ****Asset Tag Normalization Rules**** – Decide on allowed character set (alphanumeric + hyphen + underscore only, or also allow space?) and case sensitivity (upper-case only vs. case-insensitive lookup). Impact on migration of existing tags.
9. ****Label Printing Method**** – Assume Zebra printer with ZPL/EPL output, or provide fallback to generate PDF labels for any printer, or support both via config. Impact on hardware requirement and user experience.
10. ****Data Retention Policy Defaults**** – Set default retention for audit logs (e.g., 2 years), asset history (7 years after disposal), and deleted personal data (30 days). Impact on storage cost and compliance.

Each of these will be captured as a decision record in `docs/decisions/` once approved.

--- End of Technical Requirements Document (TRD) Version 1.0.0 ---